

Computação Gráfica

O Pipeline Gráfico e o OpenGL

Prof. João Paulo Gois

Universidade Federal do ABC

O OpenGL

- ▶ Frequentemente chamado de **API gráfica**, tecnicamente, é uma **especificação**.
- ▶ Define entradas e resultados; a implementação real depende dos *drivers* de hardware (NVIDIA, AMD, Intel).
- ▶ **Origem (1992)**: Gerenciado pelo consórcio *Architectural Review Board* (ARB) para padronizar o desenvolvimento gráfico de alto desempenho.
- ▶ Atualmente mantido pelo consórcio **Khronos Group**.

As Três Vertentes da API

- ▶ **Desktop:** Estável na versão 4.6 (2017). Para alto desempenho, a indústria migra para o Vulkan. No macOS, tornou-se obsoleto na versão 10.14 (mantido na v4.1).
- ▶ **OpenGL ES (*Embedded Systems*):** Para dispositivos móveis e embarcados (última especificação 3.2, em 2015). O Vulkan assumiu seu lugar (ex.: no Android), operando por meio de uma tradução (ANGLE).
- ▶ **WebGL:** Traz o OpenGL ES para o JavaScript no navegador.
 - ▶ **Sucessor:** WebGPU, padrão a partir do fim de 2025, inspirado em Vulkan/Metal e usando a linguagem WGSL.

Longevidade e Aplicabilidade

O congelamento formal de uma especificação não implica obsolescência.

O OpenGL permanece o caminho de **menor atrito** no setor produtivo para:

- ▶ Desenvolvimento de softwares CAD.
- ▶ Ferramentas de visualização científica.
- ▶ Aplicações focadas em dados e conceitos que não exigem explorar os limites computacionais e de *threads* do hardware.

Perfis de Execução e o Contexto do Curso

Durante a inicialização do contexto gráfico, o desenvolvedor deve optar por um perfil:

- ▶ **Perfil de Compatibilidade (*compatibility profile*)**: Preserva funções legadas do pipeline fixo.
- ▶ **Perfil *Core***: Suprime todas as funções obsoletas da API.

No nosso curso:

- ▶ Adotaremos o perfil ***core*** (baseado no OpenGL 4.0).
- ▶ Isso assegura compatibilidade contemporânea e garante execução até mesmo em ambientes macOS.

Integração com o Sistema Operacional

- ▶ **O escopo do OpenGL:** Ele **não** instancia janelas e **não** intercepta teclado ou mouse.
- ▶ Historicamente usava-se o GLUT (e posteriormente o freeglut).
- ▶ **A ferramenta adotada:** glfw
 - ▶ Solicita a janela, gerencia a troca de imagens (*swap buffers*) e processa eventos.
 - ▶ Evita o peso de arquiteturas mais completas e pesadas, como a SDL, que possuem outros recursos, como áudio e redes embutidas.

O OpenGL como Máquina de Estados

A API opera com base em configurações previamente definidas. Esse estado global é chamado de **contexto**.

Divisão de operações:

1. ***State-changing***: Alteram o estado global (ex: definir cor de fundo).
2. ***State-using***: Executam ações baseadas no estado configurado (ex: desenhar os vértices da memória).

No Python, a biblioteca `moderngl` abstrai a verbosidade do C (como os prefixos `gl` e `GL_`) via orientação a objetos, mas a essência do contexto permanece.

O Gargalo da Máquina de Estados

A escolha é da API, não do hardware

- ▶ **A GPU:** Possui estados (registradores) físicos.
- ▶ **O problema no OpenGL:** Permite ao programador alterar o estado **a qualquer momento**. O driver precisa rastrear isso, recompilando dados em tempo real se a alteração conflitar com o *shader*, o que gera travamentos.
- ▶ **A Solução Vulkan:** Obriga o programador a definir e congelar quase todo o estado antecipadamente (o *pipeline state object*). Elimina a flexibilidade extrema em prol de um desempenho impecável.

O Fim do Pipeline Fixo

O *immediate mode* enviava vértices um a um para a placa (gargalo de CPU/Barramento):

```
1 glBegin(GL_POLYGON);           // Inicia o objeto
2     glVertex3f(0.0, 0.0, 0.0);  // Envia o primeiro vertice
3     glVertex3f(0.0, 1.0, 0.0);  // Envia o segundo vertice
4     glVertex3f(0.0, 0.0, 1.0);  // Envia o terceiro vertice
5 glEnd();                       // Finaliza o objeto
```

Exemplo de código legado (Obsoleto)

- ▶ Este modelo foi removido no **OpenGL 3.1 (2009)**.
- ▶ Hoje alocamos a geometria definitivamente na memória da GPU por meio de *Buffer Objects*.

A Ascensão do Vulkan (2016)

- ▶ Desenvolvido *do zero* pelo Khronos Group.
- ▶ Soluciona a máquina de estado global do OpenGL, que dificultava que processadores multicore trabalhassem em paralelo ao enviar comandos de desenho.
- ▶ Congela o estado de renderização.
- ▶ Garante suporte nativo avançado para hardware moderno (como *Ray Tracing*).

A Linguagem GLSL (*OpenGL Shading Language*)

Sem o pipeline fixo, os *shaders* assumem a geometria e a cor final.

- ▶ Sintaxe em C, projetada para processamento paralelo em massa.
- ▶ Tipos nativos matemáticos (`vec3`, `vec4`, `mat4`).
- ▶ Funções nativas executadas via hardware (trigonometria, interpolação).
- ▶ **A Compilação:** Diferente do C++, a GLSL não gera binários antecipadamente. É enviada como *string* em texto simples. O driver da GPU compila de forma transparente na hora da execução (*runtime*), ajustando o código à placa do usuário.

Vulkan e a Linguagem de Shaders

- ▶ A linguagem de autoria continua sendo a **GLSL** (ou HLSL).
- ▶ No ecossistema do OpenGL, o *driver* da GPU recebe o texto simples e o compila de forma transparente em tempo de execução. O Vulkan **não** faz isso.
- ▶ O desenvolvedor deve compilar sua GLSL antecipadamente (usando compiladores externos, offline) em um formato binário intermediário chamado **SPIR-V** (.spv).
- ▶ A API do Vulkan utiliza esse binário já validado, retirando do *driver* a função de compilação de texto e eliminando travamentos.

Preparando o Ambiente: Python e Dependências

Para executar nossos códigos em OpenGL e GLSL, utilizaremos **Python 3.11** em um ambiente Conda, testado na infraestrutura do laboratório da disciplina (Windows 11, RTX 3060).

Além disso, o uso do Conda permite que cada aluno crie e gerencie seu próprio ambiente isolado sem exigir privilégios de administrador.

Passo a Passo: Criando o Ambiente Conda (1/2)

Abra o terminal (ou o *Anaconda Prompt* no Windows) e siga os passos abaixo para construir o seu ecossistema de desenvolvimento:

1. **Criar o ambiente isolado:** Vamos criar um ambiente chamado `cgcourse` travado estritamente no Python 3.11.

```
1 | conda create -n cgcourse python=3.11
```

2. **Ativar o ambiente:** Antes de instalar qualquer pacote, você precisa "entrar" nesse ambiente recém-criado.

```
1 | conda activate cgcourse
```

Passo a Passo: Criando o Ambiente Conda (2/2)

3. **Instalar as dependências:** Com o ambiente ativado, instale as bibliotecas essenciais para a infraestrutura do curso:

```
1 | pip install glfw moderngl numpy pyglm pillow
```

Para verificar se o seu ambiente está correto, inspecione a versão exata do Python e a lista de pacotes (`conda list`).

Dependências I

- ▶ `glfw`: Responsável por interagir com o sistema operacional para solicitar a criação da janela e capturar eventos de teclado e de mouse. Ela também se limita a processar a fila de eventos de entrada e a gerenciar a troca dos *buffers* de imagem (*swap buffers*).
- ▶ `moderngl`: Encapsula a API do OpenGL em Python.
- ▶ `numpy`: Biblioteca matemática para manipular matrizes e formatar blocos de memória dos vértices para a GPU.
- ▶ `pyglm`: É a ligação do GLM para Python, a mesma biblioteca comumente usada nos exemplos em C++ do OpenGL. Ela é utilizada para álgebra matricial em transformações geométricas.

Dependências II

- ▶ `pillow`: Biblioteca padrão do ecossistema Python para manipulação e processamento de imagens. Em computação gráfica, ela é essencial para carregarmos arquivos de imagem (como texturas) a partir do disco e para exportarmos renderizações (como salvar *screenshots* da nossa janela gráfica).

Nosso primeiro programa

O primeiro código que vamos executar serve para garantir que as bibliotecas e o *driver* de vídeo se comuniquem corretamente. Execute antes de tentar entendê-lo por completo. A janela ficará branca.

Nota

Neste primeiro código, ainda não há código em GLSL (nossos shaders). Ele é apenas o lado do cliente.

Código: 01_janela.py I

O Código 1 apresenta os comandos fundamentais para a inicialização de aplicações gráficas baseadas em OpenGL.

```
1  """
2  Janela, contexto OpenGL e limpeza do buffer de cor.
3  MCCC007-23 - Computação Gráfica - UFABC
4  Prof. João Paulo Gois
5
6  Executar: python 01_janela.py          Sair: ESC ou fechar a
      janela
7  """
8  import sys
9
10 try:
11     import glfw
12     import moderngl
13 except ImportError as erro:
```

Código: 01_janela.py II

```
14     sys.exit(f"FALHA: pacote ausente -> {erro}")
15
16 def erro_glfw(codigo, descricao):
17     """Substitui o aviso padrão do pyGLFW: imprime código
18     e descrição de qualquer erro do GLFW em stderr"""
19     print(f"GLFW [{codigo}]: {descricao}", file=sys.stderr)
20
21 glfw.set_error_callback(erro_glfw)
22
23 if not glfw.init():
24     sys.exit("FALHA: glfw nao inicializou")
25
26 # As dicas precisam vir ANTES de create_window
27 # que será criado junto com a janela. Depois, não há mais o
   que configurar.
28 glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 4)
29 glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 0)
```

Código: 01_janela.py III

```
30 glfw.window_hint(glfw.OPENGL_PROFILE, glfw.  
    OPENGGL_CORE_PROFILE)  
31 #Para funcionar em MacOS, é necessário habilitar o forward  
    compatibility  
32 glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, glfw.TRUE)  
33  
34 janela = glfw.create_window(800, 600, "Janela e contexto  
    OpenGL", None, None)  
35 if not janela:  
36     glfw.terminate()  
37     sys.exit("FALHA: não foi possível criar a janela")  
38  
39 # moderngl não cria contexto. Ele adota o que estiver ativo  
    na thread.  
40 # Inverter estas duas linhas quebra o programa.  
41 glfw.make_context_current(janela)  
42 ctx = moderngl.create_context()
```

Código: 01_janela.py IV

```
43
44 print(f"GL_VERSION    : {ctx.info['GL_VERSION']}")
45 print(f"GL_RENDERER   : {ctx.info['GL_RENDERER']}")
46 print(f"GL_VENDOR     : {ctx.info['GL_VENDOR']}")
47 print(f"GL (codigo)    : {ctx.version_code}")
48 print("\nESC ou feche a janela para sair.")
49
50 while not glfw.window_should_close(janela):
51     if glfw.get_key(janela, glfw.KEY_ESCAPE) == glfw.PRESS:
52         glfw.set_window_should_close(janela, True)
53
54     ctx.clear(1.0, 1.0, 1.0, 1.0)
55     glfw.swap_buffers(janela)
56     glfw.poll_events()
57
58 glfw.terminate()
```

Código: 01_janela.py V

```
59 | print("Execucao finalizada.")
```

Listing 1: codigos/01_janela.py

Entendendo os Comandos I

`glfw.init()` Inicializa a biblioteca GLFW. A função não recebe parâmetros e retorna um valor booleano indicando o sucesso da operação. Chamada obrigatória; nenhuma outra função do `glfw` pode ser invocada antes desta, exceto as de tratamento de erro.

`glfw.window_hint(constante, valor)` Recebe dois parâmetros: a constante que identifica a propriedade do contexto a ser configurada e o respectivo valor. Atuam como restrições rigorosas (*hard constraints*). Caso o *driver* não consiga satisfazê-las, a criação da janela falhará imediatamente.

- `CONTEXT_VERSION_MAJOR`, 4 e `MINOR`, 0: Exigem a criação de um contexto compatível com, no mínimo, a versão 4.0 do OpenGL.

Entendendo os Comandos II

- ▶ `OpenGL_PROFILE`, `glfw.OPENGL_CORE_PROFILE`: Restringe o contexto ao perfil *Core*, desabilitando o acesso a funções legadas do pipeline fixo (padrão adotado no curso).

`glfw.create_window(largura, altura, título, monitor, share)`

Retorna um identificador da janela instanciada, ou `None` em caso de falha.

Parâmetros chave:

- ▶ 800, 600: Dimensões em **coordenadas de tela** (podem diferir de pixels físicos em monitores *High-DPI*).
- ▶ "Janela e contexto OpenGL": Título da janela.
- ▶ `monitor = None`: Modo de janela.
- ▶ `share = None`: Não compartilha recursos com outros contextos (como VRAM, *buffers* ou texturas).

Entendendo os Comandos III

`glfw.make_context_current(janela)` Torna o contexto da janela especificada o contexto **corrente na *thread* em execução**. Toda instrução OpenGL despachada por esta *thread* será direcionada a ele.

`moderngl.create_context()` A biblioteca `moderngl` *não realiza a criação* do contexto gráfico; ela apenas encapsula e adota o contexto previamente tornado corrente na *thread*.

Atenção: A inversão da ordem entre essas duas chamadas resulta em erro fatal.

`ctx.info[...]` Dicionário do Python que armazena propriedades do *driver* e limites do *hardware* (como `GL_VENDOR` e `GL_VERSION`).

Entendendo os Comandos IV

`ctx.version_code` Número inteiro calculado pelo `moderngl`
(`maior` $\times$ 100 + `menor` $\times$ 10).

Ex.: Um contexto 4.0 resulta em 400. Coincide com o valor exigido pela diretiva `#version` dos *shaders* GLSL.

O laço principal (*main loop*) Mantém a aplicação rodando de forma iterativa enquanto `glfw.window_should_close` retornar `False`.

Operações no laço:

- ▶ `glfw.get_key(janela, glfw.KEY_ESCAPE)`: Interroga o estado instantâneo da tecla ESC.
- ▶ `ctx.clear(1.0, 1.0, 1.0, 1.0)`: Preenche o *buffer* de cor da tela com a cor RGBA especificada (intervalo de 0,0 a 1,0). A omissão disso faria com que os novos desenhos fossem renderizados sobre os antigos.

Entendendo os Comandos V

- ▶ `glfw.swap_buffers(janela)`: Efetua a troca (*swap*) entre o *frontbuffer* (visível) e o *backbuffer* (recém-processado), sem copiar dados.
- ▶ `glfw.poll_events()`: Processa a fila de eventos acumulada pelo sistema operacional. Omitir essa chamada faz o sistema achar que o processo congelou (*unresponsive*).

`glfw.terminate()` Libera todos os recursos, contextos e alocações de memória criados pelo `glfw` no encerramento.

Aqui a janela é encerrada, de fato.

O `set_window_should_close` apenas define uma flag que indica que a janela já pode ser fechada. Este tipo de instrução é útil para deixar o programa terminar o loop e também exibir uma mensagem como: *deseja encerrar o programa?*

Saída e Resultado

Saída textual do programa no terminal:

```
1      GL_VERSION    : 4.0.0 NVIDIA 610.88
2      GL_RENDERER   : NVIDIA GeForce RTX 3060/PCIe/SSE2
3      GL_VENDOR      : NVIDIA Corporation
4      GLSL           : 400
5
6      ESC ou feche a janela para sair.
7      Execucao finalizada.
```

Resultado Visual

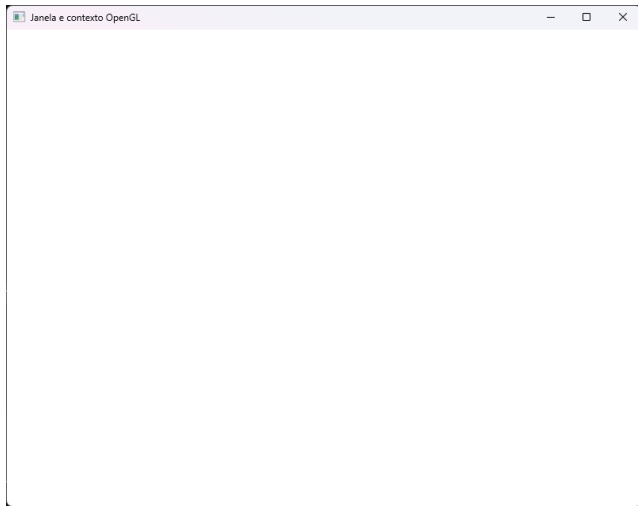


Figura: Janela gerada pela execução de `01_janela.py`

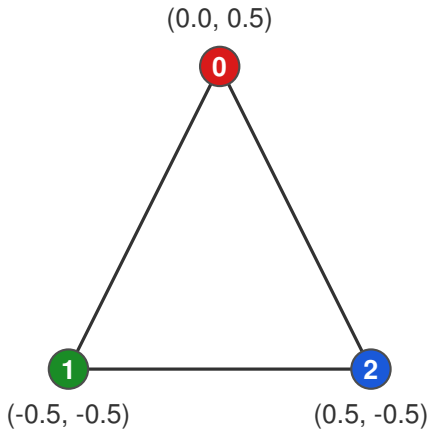
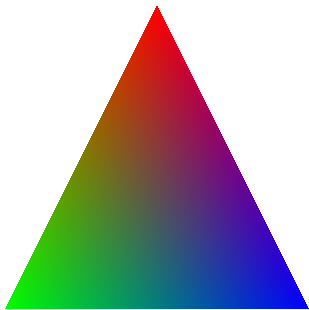
O Primeiro Triângulo em OpenGL I

Exploraremos o pipeline por meio de um exemplo que renderiza um triângulo colorido na tela.

No código a seguir, definimos primeiro os conceitos já vistos no exemplo anterior sobre criação de janelas. Neste programa, notaremos que existem duas *strings*, VERTEX_SHADER e FRAGMENT_SHADER, nas quais codificamos nossos shaders de vértice e de fragmento.

Os detalharemos após a visão geral da aplicação.

O Primeiro Triângulo em OpenGL II



Código: Triângulo I

```
1  """
2  Primeiro triangulo: VBO, VAO e o par de shaders.
3  Único programa do curso com o GLSL embutido em string.
4  A partir do 03, os shaders passam a ser gravados em arquivos
   próprios.
5
6  MCCC007-23 - Computação Gráfica - UFABC
7  Prof. João Paulo Gois
8
9  Executar: python 02_triangulo.py          Sair: ESC ou fechar a
   janela
10 """
11 import sys
12
13 import glfw
14 import moderngl
15 import numpy as np
```

Código: Triângulo II

```
16
17 def erro_glfw(codigo, descricao):
18     """Substitui o aviso padrão do pyGLFW: imprime código
19     e descrição de qualquer erro do GLFW em stderr"""
20     print(f"GLFW [{codigo}]: {descricao}", file=sys.stderr)
21
22
23 glfw.set_error_callback(erro_glfw)
24
25 if not glfw.init():
26     sys.exit("FALHA: glfw nao inicializou")
27
28 glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 4)
29 glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 0)
30 glfw.window_hint(glfw.OPENGL_PROFILE, glfw.
    OPENGL_CORE_PROFILE)
31 glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, glfw.TRUE)
```

Código: Triângulo III

```
32
33
34 janela = glfw.create_window(800, 600, "Olá triângulo", None,
    None)
35 if not janela:
36     glfw.terminate()
37     sys.exit("FALHA: não foi possível criar a janela")
38
39 glfw.make_context_current(janela)
40 ctx = moderngl.create_context()
41
42 # ----- lado da GPU
43
44 VERTEX_SHADER = """
45 #version 400 core
46
47 layout(location = 0) in vec4 vPosition;
```

Código: Triângulo IV

```
48 layout(location = 1) in vec4 vColors;
49
50 out vec4 v2fcolor;
51
52 void main() {
53     gl_Position = vPosition;
54     v2fcolor = vColors;
55 }
56 """
57
58 FRAGMENT_SHADER = """
59 #version 400 core
60
61 in vec4 v2fcolor;
62 out vec4 outfragcolor;
63
64 void main() {
```

Código: Triângulo V

```
65         outfragcolor = v2fcolor;
66     }
67     """
68
69     # O driver compila e liga os dois shaders agora, em tempo de
    execução.
70     prog = ctx.program(vertex_shader=VERTEX_SHADER,
71                        fragment_shader=FRAGMENT_SHADER)
72
73     # ----- lado da CPU
74
75     # Dados intercalados: posição (x,y,z,w) e cor (r,g,b,a) de
    cada vértice,
76     # vizinhos na memória. 'f4' é float de 32 bits.
77     # Sem dtype, o numpy usa float64 e a GPU leria lixo.
78     vertices = np.array([
```

Código: Triângulo VI

```
78     0.0,  0.5, 0.0, 1.0,    1.0, 0.0, 0.0, 1.0,    # v0 topo,
    vermelho
79     -0.5, -0.5, 0.0, 1.0,    0.0, 1.0, 0.0, 1.0,    # v1
    esquerda, verde
80     0.5, -0.5, 0.0, 1.0,    0.0, 0.0, 1.0, 1.0,    # v2
    direita, azul
81 ], dtype='f4')
82
83 # O VBO é memória bruta na VRAM.
84 # VAO diz como lê-la. '4f 4f' significa quatro floats
85 # para vPosition e quatro para vColors, nessa ordem.
86 vbo = ctx.buffer(vertices.tobytes())
87 vao = ctx.vertex_array(prog, [(vbo, '4f 4f', 'vPosition', '
    vColors')])
88
89 while not glfw.window_should_close(janela):
90     if glfw.get_key(janela, glfw.KEY_ESCAPE) == glfw.PRESS:
```

Código: Triângulo VII

```
91         glfw.set_window_should_close(janela, True)
92
93     ctx.clear(1.0, 1.0, 1.0, 1.0)
94     vao.render(moderngl.TRIANGLES)
95     glfw.swap_buffers(janela)
96     glfw.poll_events()
97
98     vao.release()
99     vbo.release()
100    prog.release()
101    glfw.terminate()
102    print("Execucao finalizada.")
```

Listing 2: codigos/02_triangulo.py

Aplicação - Preparação na CPU

Antes de iniciar o processamento na GPU, a aplicação deve especificar o formato dos dados gráficos e enviá-los à memória da placa de vídeo.

A aplicação também deve configurar as etapas programáveis do pipeline, compilar o *vertex shader* e o *fragment shader* e vincular (ligando ou *linkando*) ambos ao pipeline (usando `ctx.program(.)`) em tempo de execução.

Observações sobre Shaders

- ▶ **Vinculação (Linking):** Esta etapa é fundamental, pois a saída do *vertex shader* deve corresponder exatamente à entrada do *fragment shader*.
- ▶ **Compilação em tempo real:** A compilação dos shaders em tempo de execução permite gerar binários específicos para a GPU em uso, eliminando a dependência de hardware.

Especificando a Geometria

A geometria é especificada por um arranjo ordenado (uma matriz `numpy` denominada `vertices`), em que cada vértice é composto por um conjunto de atributos.

No exemplo, utilizamos **dados intercalados** (*interleaved data*): uma posição 4D e uma cor RGBA dispostas sequencialmente na memória.

Por que usar vetores 4D em um cenário 2D?

Vetores de 4 componentes (x, y, z, w)

Mesmo operando em 2D, o OpenGL adota esse formato padrão pelos seguintes motivos:

- ▶ **Compatibilidade:** A API trata o 2D como um caso especial do 3D em que a profundidade é constante ($z = 0$).
- ▶ **Cálculo de Projeções:** A quarta coordenada (w) é uma ferramenta matemática essencial (coordenada homogênea) para realizar projeções.
- ▶ **Padronização de Dados:** Estruturas de 4 elementos unificam o processamento. O mesmo vetor serve para posições espaciais e cores (r, g, b, a).

Memória da GPU: VBO e VAO I

Para ser utilizado pelo pipeline, o bloco de bytes precisa ser alocado na VRAM da GPU por meio de um **Vertex Buffer Object (VBO)** (`ctx.buffer(.)`).

Como o VBO armazena apenas memória bruta, a sua estruturação — no nosso caso, '4f 4f' indicando 4 *floats* para a posição e 4 para a cor — deve ser definida em um **Vertex Array Object (VAO)** (`ctx.vertex_array(.)`).

O VAO instrui a placa de vídeo sobre como fatiar e mapear a memória do VBO para as portas de entrada corretas do *vertex shader*.

Memória da GPU: VBO e VAO II

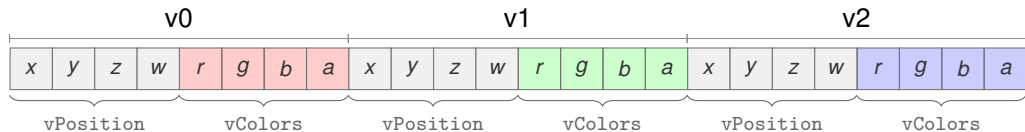


Figura: O conteúdo do VBO do primeiro triângulo: 24 *floats* de 4 bytes, com posição e cor de cada vértice lado a lado (32 bytes por vértice). O formato '4f 4f' diz ao VAO que os quatro primeiros *floats* de cada vértice vão para `vPosition` e os quatro seguintes para `vColors`.

Comando de Renderização

O tipo de primitiva formada a partir dos vértices é determinado pelo comando de renderização (`vao.render(.)`). Neste caso, utilizamos `moderngl.TRIANGLES`.

É a execução deste comando que efetivamente engatilha toda a cascata do pipeline gráfico na GPU.

Primitivas do OpenGL suportadas

- ▶ `GL_POINTS`: cada vértice é um ponto.
- ▶ `GL_LINES`: cada grupo de dois vértices forma uma reta.
- ▶ `GL_LINE_STRIP`: vértices formam uma polilinha.
- ▶ `GL_LINE_LOOP`: forma uma polilinha fechada.
- ▶ `GL_TRIANGLES`: cada conjunto de 3 vértices forma um triângulo.
- ▶ `GL_TRIANGLE_STRIP`: faixa de triângulos com arestas compartilhadas (economiza memória).
- ▶ `GL_TRIANGLE_FAN`: leque de triângulos compartilhando o 1.º vértice.

Representação Visual das Primitivas

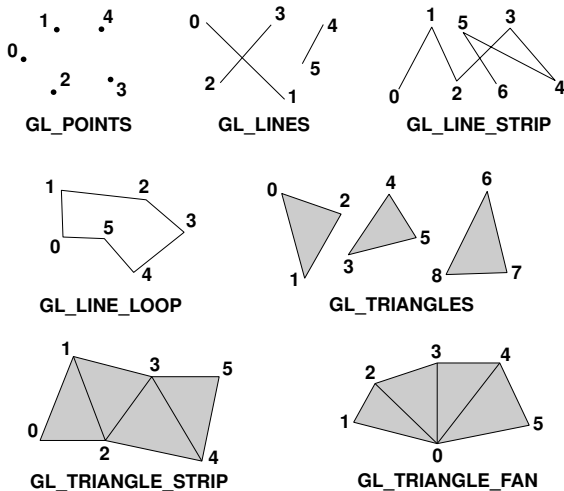
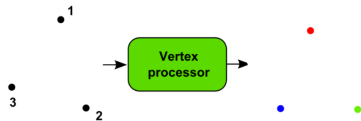


Figura: Principais primitivas do OpenGL.

Vertex Shader

O *vertex shader* processa cada vértice individualmente e em paralelo.

Sua entrada é composta pelos atributos fornecidos pela aplicação à CPU (via VBO) de acordo com o formato (via VAO).



Vertex Shader em Detalhes (1/2)

`#version 400 core`: Identifica a especificação GLSL 4.0 no perfil *Core*. Deve ser estritamente a primeira linha do código.

`layout(location = X) in vec4...`: Define variáveis de entrada (*in*) do VBO. O qualificador numera as portas de entrada, indicando à GPU exatamente onde referenciar posições e cores.

`out vec4 v2fcolor`: Declara um atributo de saída (*out*). Seu conteúdo será enviado às próximas etapas do *pipeline*, alimentando o *fragment shader*.

Vertex Shader em Detalhes (2/2)

`void main()`: Função chamada para **cada vértice**. Execução massivamente paralela (sem estados compartilhados entre vértices vizinhos).

`gl_Position = ...`: Variável embutida **obrigatória**. Define a posição final 4D (x, y, z, w) do vértice (espaço de recorte).

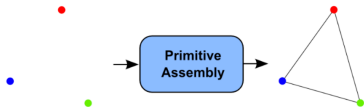
`v2fcolor = vColors`: O atributo de saída simplesmente repassa a cor original do vértice adiante.

Montagem de Primitivas (Primitive Assembly)

Esta etapa recebe os vértices calculados pelo *vertex shader* e os conecta estruturalmente.

Como especificamos `moderngl.TRIANGLES` na chamada do `render()`, a GPU formará um triângulo a cada três vértices.

Neste estágio definem-se as combinações topológicas (conexão entre os vértices), e não as arestas geométricas per se.

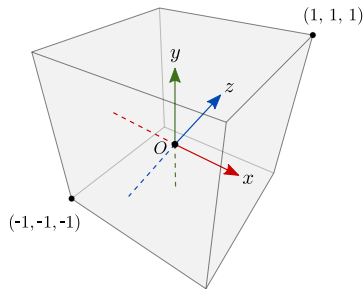


Recorte (Clipping)

Geometrias (ou porções) que caem fora do volume de visão canônico são descartadas, evitando processamento inútil.

Esse volume corresponde a um cubo espacial $2 \times 2 \times 2$, no intervalo de -1 a 1 em todos os eixos coordenados. Primitivas parcialmente dentro são reconstruídas.

Em nosso exemplo (coordenadas $x, y = \pm 0,5$), tudo já está dentro da faixa visível.

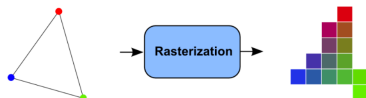


Rasterização

As primitivas passam pelo rasterizador (hardware fixo), gerando **fragmentos**.

Um fragmento é um *pixel em potencial*. Cada fragmento é descrito por sua posição (x, y, z) e seus **atributos interpolados**.

O hardware calcula as coordenadas baricêntricas para preencher o triângulo interpolando (degradê) as cores vermelho, verde e azul que enviamos.

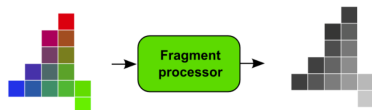


Fragment Shader

É o segundo estágio programável do pipeline. Processa cada fragmento após a rasterização.

Sua entrada corresponde às variáveis declaradas como `out` no *vertex shader*, agora interpoladas.

No nosso exemplo, ele atua como um *pass-through* (apenas recebe a cor e a envia para a saída sem modificações).



Fragment Shader em Detalhes

`in vec4 v2fcolor`: Recebe a cor do estágio anterior. O nome e o tipo devem ser idênticos ao atributo `out` do *vertex shader*.

`out vec4 outfragcolor`: Variável de saída final. Define a cor do fragmento (formato RGBA) que será avaliada para ir à tela.

`void main()`: Chamada para **cada fragmento**.

`outfragcolor = v2fcolor`: Repassa a cor interpolada (`v2fcolor`) para a saída, definindo o visual final do pixel.

O Ciclo de Renderização e a Exibição da Imagem

Após o *fragment shader*, a imagem final fica retida temporariamente na memória interna da GPU (*backbuffer*). Para exibir continuamente o que está sendo desenhado, utilizamos um laço de repetição:

1. Limpamos, a cada *frame*, a janela com uma cor fixa: `ctx.clear(1.0, 1.0, 1.0, 1.0)`.
2. O comando de renderização repopula a memória visual na GPU.
3. O comando `glfw.swap_buffers(janela)` troca os *buffers*, transferindo o que foi pintado pelo shader para o monitor do usuário.

Resultado Final: O Primeiro Triângulo

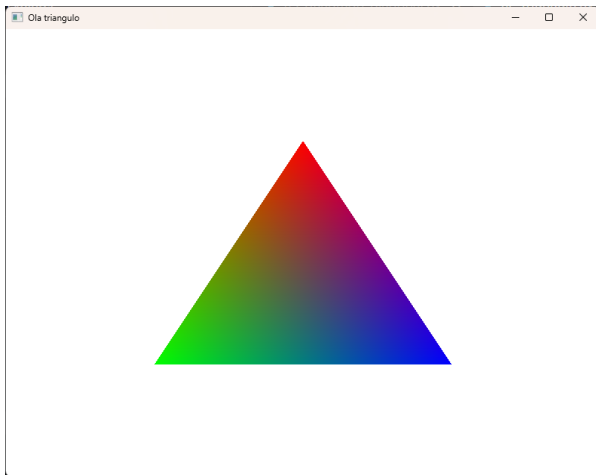


Figura: Saída do programa 02_triângulo.py.

Memória na CPU vs. Memória na GPU

- ▶ **CPU (Memória RAM):** Arranjos criados com `numpy`, gerenciados pelo *Garbage Collector* do Python.
- ▶ **GPU (Memória VRAM):** Objetos do OpenGL (VBO, VAO, Shaders), gerenciados pelo *driver* de vídeo.

Ao instanciar um VBO, os dados são copiados da RAM para a VRAM:

```
1 | vbo = ctx.buffer(vertices.tobytes())
```

```
2 |
```

A partir desse momento, a placa de vídeo usa a **sua própria cópia** para desenhar o quadro.

Otimizando a Memória da CPU

A cópia na RAM pode (e muitas vezes deve) ser descartada.

Em malhas com milhares ou milhões de vértices, manter o numpy array na memória dobra o uso global do sistema sem necessidade.

```
1 | del vertices    # Remove a referencia do Python
2 |
```

Quando NÃO devemos descartar?

- ▶ Quando a geometria for dinâmica (animações, física feita na CPU).
- ▶ Quando precisarmos sobrescrever o buffer em tempo de execução utilizando o comando `vbo.write(...)`.

Limpendo a Memória da GPU

A GPU exige limpeza manual!

- ▶ O `moderngl` **não apaga** os objetos da VRAM automaticamente quando a variável Python morre.
- ▶ Criar buffers em um loop sem liberar recursos causa um vazamento rápido de VRAM.

```
1 vao.release()      # Recomendado: ordem inversa da criacao
2 vbo.release()
3 prog.release()
4 glfw.terminate()
5
```

Nota: Fazer isso ao encerrar o *script* é redundante, mas cria o hábito disciplinar essencial para quando formos carregar e descarregar malhas 3D dinamicamente!

Removendo o GLSL de dentro do Python

- ▶ No exemplo anterior, o código GLSL era uma *string* dentro do Python.
- ▶ Inconveniente para programas maiores com *shaders* complexos ou com diversos *shaders*.
- ▶ **Solução:** Armazenar os *shaders* em arquivos-fonte separados (pasta `shaders/`).
- ▶ Extensões comuns: `.vert` e `.frag` (ou `.glsl`).

Vantagens da separação:

- ▶ O GLSL passa a ser tratado como linguagem na IDE (realce de sintaxe, indentação, linhas de erro).
- ▶ O *driver* continua recebendo uma *string*, como leitura de arquivo.

Leitura dos Shaders e Caminhos Relativos

Para carregar os arquivos sem depender do diretório de onde o terminal foi aberto, utilizamos caminhos relativos ao *script* atual:

Caminho relativo ao arquivo, não ao terminal

A leitura com `Path(__file__).parent` fornece o caminho do script atual, isolando o diretório raiz esperado. Isso evita a exceção `FileNotFoundError`.

```
1 from pathlib import Path
2
3 SHADERS = Path(__file__).parent / "shaders"
4
5 prog = ctx.program(
6     vertex_shader=(SHADERS / "basico.vert").read_text(
7         encoding="utf-8"),
8     fragment_shader=(SHADERS / "basico.frag").read_text(
9         encoding="utf-8"),
10 )
```

O Par de Shaders Básicos I

```
1 #version 400 core
2 // Vertex shader de repasse.
3 // Nao transforma a geometria, apenas a encaminha.
4 // Usado por 03_triangulo.py, 04_quadrado.py e 05
   _quadrado_diagonal.py.
5
6 layout(location = 0) in vec4 vPosition;
7 layout(location = 1) in vec4 vColors;
8
9 out vec4 v2fcolor;
10
11 void main() {
12     gl_Position = vPosition;
13     v2fcolor = vColors;
14 }
```

Listing 3: codigos/shaders/basico.vert

O Par de Shaders Básicos II

```
1  #version 400 core
2  // Fragment shader de repasse.
3  // Devolve a cor ja interpolada pelo rasterizador.
4
5  in vec4 v2fcolor;
6  out vec4 outfragcolor;
7
8  void main() {
9      outfragcolor = v2fcolor;
10 }
```

Listing 4: codigos/shaders/basico.frag

Atenção

Como `basico.vert` e `basico.frag` serão usados em vários programas, cuidado ao editá-los.

Múltiplos VBOs e Dados Não Intercalados

- ▶ **Dados intercalados:** Como no exemplo atual, posição e cor dividem o mesmo arranjo e o mesmo VBO.
- ▶ **Dados não intercalados:** Atributos chegam de listas distintas na aplicação.
- ▶ Muito comum ao carregar malhas 3D (posições, normais, texturas vêm separadas).

O **Vertex Array Object (VAO)** organiza e conecta múltiplos VBOs a um único estado. Passamos uma lista de tuplas que une a origem física do dado à variável no *shader*.

Código: Triângulo com 2 VBOs I

```
1  """
2  Triângulo com dados NÃO intercalados.
3  Um VBO por atributo.
4  Primeiro programa com shaders em arquivos separados (shaders
   /basico.*).
5
6  MCCC007-23 - Computação Gráfica - UFABC
7  Prof. João Paulo Gois
8
9  Executar: python 03_triangulo.py          Sair: ESC ou fechar a
   janela
10 """
11 import sys
12 from pathlib import Path
13
14 import glfw
15 import moderngl
```

Código: Triângulo com 2 VBOs II

```
16 import numpy as np
17
18 # Caminho relativo ao arquivo.
19 # Não ao diretório a partir do qual o Python foi chamado.
20 SHADERS = Path(__file__).parent / "shaders"
21
22 def erro_glwf(codigo, descricao):
23     """Substitui o aviso padrão do pyGLFW: imprime código
24     e descrição de qualquer erro do GLFW em stderr"""
25     print(f"GLFW [{codigo}]: {descricao}", file=sys.stderr)
26
27
28 glfw.set_error_callback(erro_glwf)
29
30 if not glfw.init():
31     sys.exit("FALHA: glfw nao inicializou")
32
```

Código: Triângulo com 2 VBOs III

```
33 glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 4)
34 glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 0)
35 glfw.window_hint(glfw.OPENGL_PROFILE, glfw.
    OPENGGL_CORE_PROFILE)
36 glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, glfw.TRUE)
37
38 janela = glfw.create_window(800, 600, "Olá triângulo (dois
    VBOs)", None, None)
39 if not janela:
40     glfw.terminate()
41     sys.exit("FALHA: não foi possível criar a janela")
42
43 glfw.make_context_current(janela)
44 ctx = moderngl.create_context()
45
46 # Para o driver, o shader continua sendo uma string.
47 # Ela só passou a ser lida a partir de arquivos.
```

Código: Triângulo com 2 VBOs IV

```
48 prog = ctx.program(  
49     vertex_shader=(SHADERS / "basico.vert").read_text(  
        encoding="utf-8"),  
50     fragment_shader=(SHADERS / "basico.frag").read_text(  
        encoding="utf-8"),  
51 )  
52  
53 # Arranjos independentes na CPU.  
54 posicoes = np.array([  
55     0.0,  0.5, 0.0, 1.0,    # v0 topo  
56     -0.5, -0.5, 0.0, 1.0,    # v1 esquerda  
57     0.5, -0.5, 0.0, 1.0,    # v2 direita  
58 ], dtype='f4')  
59  
60 cores = np.array([  
61     1.0, 0.0, 0.0, 1.0,    # v0 vermelho  
62     0.0, 1.0, 0.0, 1.0,    # v1 verde
```

Código: Triângulo com 2 VBOs V

```
63         0.0, 0.0, 1.0, 1.0,      # v2 azul
64     ], dtype='f4')
65
66     vbo_pos = ctx.buffer(posicoes.tobytes())
67     vbo_cor = ctx.buffer(cores.tobytes())
68
69     # Um VAO pode reunir vários VBOs.
70     vao = ctx.vertex_array(prog, [
71         (vbo_pos, '4f', 'vPosition'),
72         (vbo_cor, '4f', 'vColors'),
73     ])
74
75     while not glfw.window_should_close(janela):
76         if glfw.get_key(janela, glfw.KEY_ESCAPE) == glfw.PRESS:
77             glfw.set_window_should_close(janela, True)
78
79         ctx.clear(1.0, 1.0, 1.0, 1.0)
```

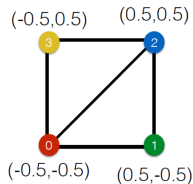
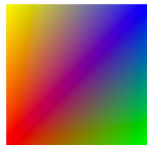
Código: Triângulo com 2 VBOs VI

```
80     vao.render(moderngl.TRIANGLES)
81     glfw.swap_buffers(janela)
82     glfw.poll_events()
83
84     vao.release()
85     vbo_pos.release()
86     vbo_cor.release()
87     prog.release()
88     glfw.terminate()
89     print("Execucao finalizada.")
```

Listing 5: codigos/03_triângulo.py

Reaproveitando Vértices (EBO)

- ▶ O OpenGL renderiza apenas pontos, retas e triângulos. Outros polígonos são composições.
- ▶ Definir um quadrado com 2 triângulos isolados requer 6 vértices (2 duplicados na diagonal).
- ▶ **Desperdício:** Repetir vértices gasta memória e gera problemas topológicos.
- ▶ **Solução:** O *Element Buffer Object* (EBO) armazena índices informando a ordem de leitura dos vértices únicos.



Código: Quadrado com EBO I

```
1  """
2  Quadrado com EBO: quatro vertices, dois triangulos, seis
   indices.
3  MCCC007-23 - Computacao Grafica - UFABC
4
5  Executar: python 04_quadrado.py          Sair: ESC ou fechar a
   janela
6  """
7  import sys
8  from pathlib import Path
9
10 import glfw
11 import moderngl
12 import numpy as np
13
14 SHADERS = Path(__file__).parent / "shaders"
15
```

Código: Quadrado com EBO II

```
16 def erro_glfw(codigo, descricao):
17     """Substitui o aviso padrão do pyGLFW: imprime código
18     e descrição de qualquer erro do GLFW em stderr"""
19     print(f"GLFW [{codigo}]: {descricao}", file=sys.stderr)
20
21
22 glfw.set_error_callback(erro_glfw)    # antes de glfw.init(),
    de proposito
23
24 if not glfw.init():
25     sys.exit("FALHA: glfw nao inicializou")
26
27 glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 4)
28 glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 0)
29 glfw.window_hint(glfw.OPENGL_PROFILE, glfw.
    OPENGL_CORE_PROFILE)
30 glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, glfw.TRUE)
```

Código: Quadrado com EBO III

```
31
32
33 janela = glfw.create_window(600, 600, "Quadrado com EBO",
    None, None)
34 if not janela:
35     glfw.terminate()
36     sys.exit("FALHA: nao foi possivel criar a janela")
37
38 glfw.make_context_current(janela)
39 ctx = moderngl.create_context()
40
41 prog = ctx.program(
42     vertex_shader=(SHADERS / "basico.vert").read_text(
43         encoding="utf-8"),
44     fragment_shader=(SHADERS / "basico.frag").read_text(
45         encoding="utf-8"),
46 )
```

Código: Quadrado com EBO IV

```
45
46 vertices = np.array([
47     -0.5, -0.5, 0.0, 1.0,    1.0, 0.0, 0.0, 1.0,    # v0
        vermelho
48     0.5, -0.5, 0.0, 1.0,    0.0, 1.0, 0.0, 1.0,    # v1 verde
49     0.5,  0.5, 0.0, 1.0,    0.0, 0.0, 1.0, 1.0,    # v2 azul
50     -0.5,  0.5, 0.0, 1.0,    1.0, 1.0, 0.0, 1.0,    # v3
        amarelo
51 ], dtype='f4')
52
53 # Dois triangulos pela diagonal 0-2. Os indices sao inteiros
54 SEM sinal:
55 # o arranjo de elementos do OpenGL e GL_UNSIGNED_INT.
56 indices = np.array([0, 1, 2,  2, 3, 0], dtype='u4')
57
58 vbo = ctx.buffer(vertices.tobytes())
59 ebo = ctx.buffer(indices.tobytes())
```

Código: Quadrado com EBO V

```
59
60 vao = ctx.vertex_array(
61     prog, [(vbo, '4f 4f', 'vPosition', 'vColors')],
62     index_buffer=ebo
63 )
64 while not glfw.window_should_close(janela):
65     if glfw.get_key(janela, glfw.KEY_ESCAPE) == glfw.PRESS:
66         glfw.set_window_should_close(janela, True)
67
68     ctx.clear(1.0, 1.0, 1.0, 1.0)
69     vao.render(moderngl.TRIANGLES)
70     glfw.swap_buffers(janela)
71     glfw.poll_events()
72
73 vao.release()
74 ebo.release()
```

Código: Quadrado com EBO VI

```
75 vbo.release()  
76 prog.release()  
77 glfw.terminate()  
78 print("Execucao finalizada.")
```

Listing 6: codigos/04_quadrado.py

Manipulação Dinâmica: Estados e Teclado I

Agora o programa precisa memorizar informações (estados), como qual diagonal ou fundo está ativo. Para alterá-los, usamos interações via teclado.

Consulta Direta (`glfw.get_key`)

Responde: “*A tecla está pressionada agora?*”

- ▶ **Uso:** Ações contínuas (segurar para acelerar, mover personagem).
- ▶ **Problema:** Em ações discretas, alterna incontrolavelmente 60x por segundo.

Manipulação Dinâmica: Estados e Teclado II

Eventos por Callback

Responde: *“O evento de pressionar aconteceu?”*

- ▶ **Uso:** Ações discretas (alternar cor, fechar, abrir arquivo).
- ▶ **Vantagem:** Registra todos os eventos em fila e executa apenas uma vez por toque, evitando alternâncias múltiplas indevidas.

Atenção: Ambos dependem de `glfw.poll_events()` para esvaziar a fila do sistema operacional e atualizar os estados.

Quadrado com Diagonal Dinâmica I

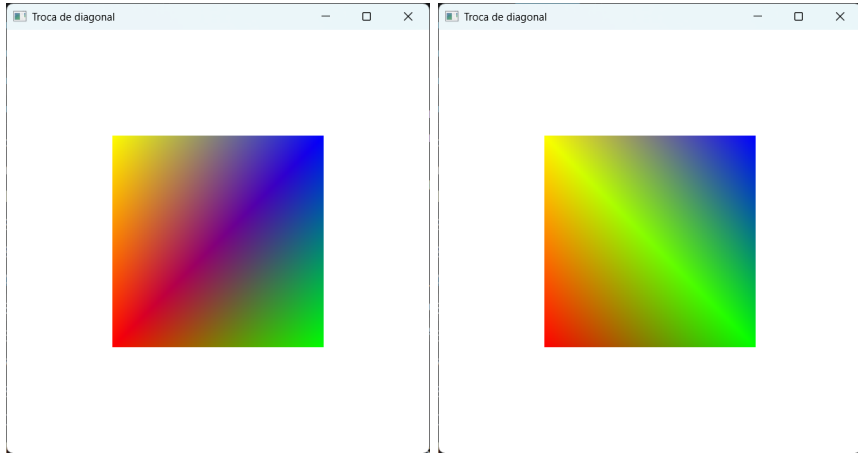


Figura: Troca da diagonal.

Código: Quadrado com Diagonal Dinâmica I

```
1  """
2  Quadrado com troca de diagonal em tempo de execucao.
3  Primeiro programa com estado: qual diagonal e qual fundo
   estao ativos.
4  MCCC007-23 - Computacao Grafica - UFABC
5
6  Executar: python 05_quadrado_diagonal.py
7  Teclas:   ESPACO alterna a diagonal | D alterna o fundo |
           ESC sai
8  """
9  import sys
10 from pathlib import Path
11
12 import glfw
13 import moderngl
14 import numpy as np
15
```

Código: Quadrado com Diagonal Dinâmica II

```
16 SHADERS = Path(__file__).parent / "shaders"
17
18 VERTICES = np.array([
19     -0.5, -0.5, 0.0, 1.0,    1.0, 0.0, 0.0, 1.0,    # v0
        vermelho
20     0.5, -0.5, 0.0, 1.0,    0.0, 1.0, 0.0, 1.0,    # v1 verde
21     0.5,  0.5, 0.0, 1.0,    0.0, 0.0, 1.0, 1.0,    # v2 azul
22     -0.5,  0.5, 0.0, 1.0,    1.0, 1.0, 0.0, 1.0,    # v3
        amarelo
23 ], dtype='f4')
24
25 # Os mesmos quatro vertices, triangulados de duas maneiras.
    Muda so a ordem
26 # de leitura; o VBO nao e tocado.
27 DIAGONAL_02 = np.array([0, 1, 2,  2, 3, 0], dtype='u4')
28 DIAGONAL_13 = np.array([0, 1, 3,  1, 2, 3], dtype='u4')
29
```

Código: Quadrado com Diagonal Dinâmica III

```
30 BRANCO = (1.0, 1.0, 1.0, 1.0)
31 PRETO = (0.0, 0.0, 0.0, 1.0)
32
33
34 def erro_glfw(codigo, descricao):
35     """Sem este callback, a razão real de uma falha do GLFW
36     e descartada."""
37     print(f"GLFW [{codigo}]: {descricao}", file=sys.stderr)
38
39 glfw.set_error_callback(erro_glfw)    # antes de glfw.init(),
40                                       de proposito
41
42 if not glfw.init():
43     sys.exit("FALHA: glfw não inicializou")
44
45 glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 4)
```

Código: Quadrado com Diagonal Dinâmica IV

```
45 glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 0)
46 glfw.window_hint(glfw.OPENGL_PROFILE, glfw.
    OPENGL_CORE_PROFILE)
47 glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, glfw.TRUE)
48 # Sem a dica acima, o macOS recusa qualquer contexto 3.2+.
49
50 janela = glfw.create_window(600, 600, "Troca de diagonal",
    None, None)
51 if not janela:
52     glfw.terminate()
53     sys.exit("FALHA: nao foi possivel criar a janela")
54
55 glfw.make_context_current(janela)
56 glfw.swap_interval(1)
57 ctx = moderngl.create_context()
58
59 prog = ctx.program(
```

Código: Quadrado com Diagonal Dinâmica V

```
60     vertex_shader=(SHADERS / "basico.vert").read_text(  
        encoding="utf-8"),  
61     fragment_shader=(SHADERS / "basico.frag").read_text(  
        encoding="utf-8"),  
62 )  
63  
64 vbo = ctx.buffer(VERTICES.tobytes())  
65 ebo = ctx.buffer(DIAGONAL_02.tobytes())  
66 vao = ctx.vertex_array(  
67     prog,  
68     [(vbo, '4f 4f', 'vPosition', 'vColors')],  
69     index_buffer=ebo,  
70 )  
71  
72 # O estado que os exemplos anteriores nao tinham.  
73 diagonal_alternativa = False  
74 modo_noite = False
```

Código: Quadrado com Diagonal Dinâmica VI

```
75
76
77 def tecla(window, key, scancode, action, mods):
78     """Chamada pelo GLFW uma vez por evento de teclado.
79
80     Os argumentos sao fixos, e o estado do programa nao esta
81     entre eles.
82     Para altera-lo, a funcao declara as variaveis como
83     global; sem essa
84     declaracao, a atribuicao criaria variaveis locais, e o
85     estado do
86     programa nao mudaria.
87
88     Alternar estado exige o INSTANTE da transicao. Com glfw.
89     get_key,
90     segurar a tecla inverteria o valor a cada quadro.
91     """
```

Código: Quadrado com Diagonal Dinâmica VII

```
88     global diagonal_alternativa, modo_noite
89     if action != glfw.PRESS:
90         return
91
92     if key == glfw.KEY_ESCAPE:
93         glfw.set_window_should_close(window, True)
94     elif key == glfw.KEY_D:
95         modo_noite = not modo_noite
96     elif key == glfw.KEY_SPACE:
97         diagonal_alternativa = not diagonal_alternativa
98         # Reescreve os 24 bytes do EBO já alocado, sem
99         recriar buffer nem VAO.
100         novos = DIAGONAL_13 if diagonal_alternativa else
101         DIAGONAL_02
102         ebo.write(novos.tobytes())
```

Código: Quadrado com Diagonal Dinâmica VIII

```
103 glfw.set_key_callback(janela, tecla)
104
105 while not glfw.window_should_close(janela):
106     ctx.clear(*(PRETO if modo_noite else BRANCO))
107     vao.render(moderngl.TRIANGLES)
108     glfw.swap_buffers(janela)
109     glfw.poll_events()
110
111 for recurso in (vao, ebo, vbo, prog):
112     recurso.release()
113 glfw.terminate()
114 print("Execucao finalizada.")
```

Listing 7: codigos/05_quadrado_diagonal.py

O Gargalo de Memória (CPU vs GPU) I

O custo de comunicação

- ▶ A cópia da RAM para a VRAM passa pelo barramento (PCIe), o que a torna mais lenta do que o acesso da GPU à própria VRAM.
- ▶ Reescrever um *buffer* que a GPU ainda está usando pode obrigar a CPU e a GPU a esperarem uma pela outra.
- ▶ Enviar grandes volumes de dados a cada quadro deixa a GPU ociosa, à espera deles.

O Gargalo de Memória (CPU vs GPU) II

Regra prática: enviar uma vez, escolher depois

- ▶ As duas diagonais do quadrado cabem no mesmo EBO; basta desenhar a partir de `first=0` ou `first=6`.
- ▶ Quando os dados mudam de fato a cada quadro (partículas, por exemplo), há mecanismos próprios, como os *buffers* persistentes do OpenGL 4.4.

O Operador Asterisco (*) em Python

Desempacotamento vs. Ponteiros

No comando `ctx.clear(*(PRETO if modo_noite else BRANCO))`, o `*` **não é um ponteiro** de memória, como no C/C++. Ele é o **operador de desempacotamento**.

A função `clear(.)` exige quatro valores separados (RGBA). Como nossas cores são tuplas, o `*` as divide:

- ▶ **Sem `*`:** Envia uma única tupla (gera um erro de execução).
- ▶ **Com `*`:** Desempacota `(0,0,0,1)` em `0, 0, 0, 1` como argumentos separados.

O Conceito de Swizzling no GLSL

Diferente do Python ou do C, no GLSL podemos reordenar e acessar múltiplos componentes de um vetor usando sequências de letras. Isso é chamado de ***Swizzling***.

Podem ser referenciados de 3 formas equivalentes, dependendo do contexto lógico:

- ▶ **Coordenadas Espaciais:** x, y, z, w
- ▶ **Canais de Cores:** r, g, b, a
- ▶ **Coordenadas de Textura:** s, t, p, q

Exemplos e Regras do Swizzling

Considerando `vec4 cor = vec4(1.0, 0.5, 0.2, 1.0);`

- ▶ **Extração:** `vec3 rgb = cor.rgb;`
- ▶ **Inversão:** `vec4 invertida = cor.abgr;`
- ▶ **Repetição:** `vec4 cinza = cor.rrrr;`
- ▶ **Atribuição Mascarada:** `cor.bg = vec2(1.0, 0.0);`
- ▶ **Permutação Cruzada:** `v1.zyx = v2.xzy;`

Restrição na Atribuição

No lado esquerdo da igualdade, **não é permitido repetir componentes**. Um comando como `v1.xx = vec2(1.0, 2.0);` gera erro de compilação, pois a GPU não sabe qual dos valores gravar na porta `x`.

Variáveis Uniformes I

Além dos dados de vértices (VBOs), a aplicação também pode enviar parâmetros globais para a GPU.

- ▶ **Uniform Variables:** Contêm dados de leitura que permanecem idênticos para todos os vértices/fragmentos de uma mesma chamada de desenho (*draw call*).
- ▶ Permitem repassar os estados lógicos dos objetos (escala, cor global, tempo) ao *pipeline*.
- ▶ Persistem na memória da GPU como "estado do programa" até serem sobrescritas explicitamente. Não precisam ser reenviadas se não houver mudança.

Exemplo: Uso de Uniformes I

1. Declarando no GLSL:

```
1  #version 400 core
2  layout(location = 0) in vec4 vPosition;
3
4  uniform float u_escala;
5  uniform vec2  u_deslocamento;
6  uniform vec4  u_cor;
7
8  void main() {
9      // Uso livre nas equacoes matematicas
10 }
```

Exemplo: Uso de Uniformes II

2. Injetando dados do Python para o GLSL:

```
1  # Definimos valores no Python
2  escala = 1.5
3  desloc = (0.2, -0.1)
4  cor_verm = (1.0, 0.0, 0.0, 1.0)
5
6  # Injetamos como dicionario no objeto do programa
7  prog['u_escala'].value = escala
8  prog['u_deslocamento'].value = desloc
9  prog['u_cor'].value = cor_verm
10
11 # Todos os fragmentos desta chamada usarao estes valores
12 vao.render(moderngl.TRIANGLES)
```

Suavização de Bordas (Anti-Aliasing) I

As formas matemáticas são contínuas, mas a tela é uma grade discreta de pixels. O rasterizador toma decisões binárias ao avaliar o centro do pixel, gerando serrilhado (*aliasing*) e tremores em objetos animados.

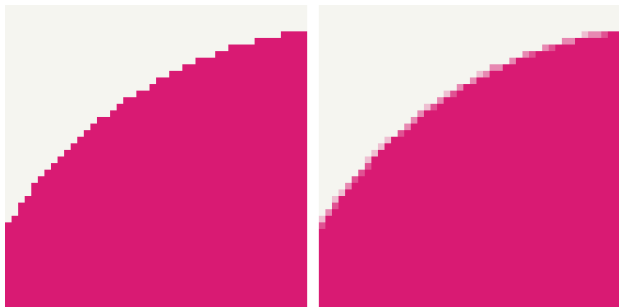


Figura: À esquerda: uma amostra (serrilhado binário). À direita: quatro amostras que produzem uma transição suave.

Suavização de Bordas (Anti-Aliasing) II

Para contornar, usamos **Multisample Anti-Aliasing (MSAA)**:

```
glfw.window_hint(GLFW_SAMPLES, 4)
```

- ▶ **Custo de Memória:** Aloca memória adicional para rastrear a profundidade e a cobertura de 4 subamostras por pixel.
- ▶ **Economia de Processamento:** O hardware **não** gera 4 fragmentos. O *fragment shader* é executado apenas uma vez por pixel coberto.

No final, a cor calculada é misturada proporcionalmente à quantidade de subamostras que o polígono efetivamente cobriu (em tons intermediários).

Exemplo: Sr. Coração

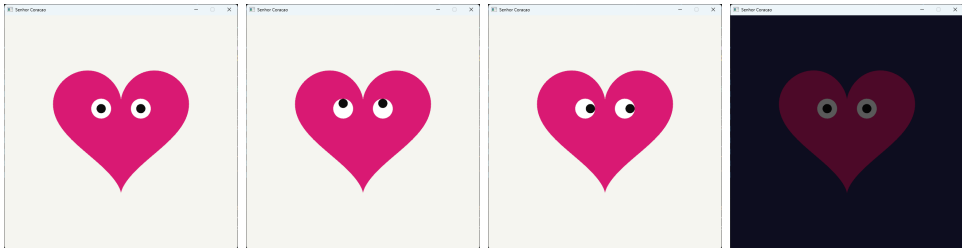


Figura: Sr. Coração.

Sr. Coração: Reaproveitando Geometrias

No exemplo do *Sr. Coração*, veremos como reaproveitar uma única geometria na memória para desenhá-la várias vezes na tela, variando sua posição, cor, tamanho e função.

A Interação (Teclado):

- ▶ As setas movem as pupilas e a tecla D alterna o modo dia/noite.
- ▶ **Importante:** As teclas **não destroem nem reescrevem o *buffer*** na VRAM.
- ▶ Elas apenas alteram variáveis na CPU, que são enviadas à GPU como **variáveis uniformes** no quadro seguinte, atualizando a imagem de forma extremamente eficiente.

Sr. Coração: Estrutura da Cena

Toda a personagem é definida por apenas dois conjuntos de vértices na VRAM: a curva do coração e um círculo de raio unitário. O laço dispara cinco chamadas de desenho por quadro:

Desenho	VAO	u_escala	u_deslocamento
corpo	vao_coracao	0,620	(0; 0)
olho esquerdo	vao_circulo	0,085	(-0,17; 0,20)
pupila esq.	vao_circulo	0,040	(-0,17; 0,20) + olhar
olho direito	vao_circulo	0,085	(+0,17; 0,20)
pupila dir.	vao_circulo	0,040	(+0,17; 0,20) + olhar

Reaproveitamento de Geometria e Atualização de Estados I

O `vao_circulo` é desenhado quatro vezes. Todas as chamadas percorrem os mesmos vértices; só mudam os valores dos uniformes.

```
1 def desenhar(vao, escala, deslocamento, cor):
2     """Carrega os tres uniformes daquele desenho e dispara a
3     chamada."""
4     ajustar('u_escala', escala)
5     ajustar('u_deslocamento', deslocamento)
6     ajustar('u_cor', cor)
7     vao.render(moderngl.TRIANGLE_FAN)
```

- ▶ `ajustar` só escreve no uniforme se ele existir no *shader*: um uniforme não usado é descartado na compilação.
- ▶ Uma malha na memória, várias cópias na tela.

Reaproveitamento de Geometria e Atualização de Estados II

Escopos de atualização no mesmo laço

- ▶ `u_atenuacao`: carregado **uma única vez por quadro**. Os cinco desenhos o leem porque ele permanece no estado do programa.
- ▶ `u_escala`, `u_deslocamento`, `u_cor`: reescritos **antes de cada chamada** de desenhar.

Reaproveitamento de Geometria e Atualização de Estados III

E o *instancing*?

- ▶ Aqui há cinco chamadas de desenho, uma por cópia.
- ▶ Com milhares de cópias (árvores, NPCs), usa-se a **renderização instanciada: uma única chamada** desenha todas, e o *vertex shader* distingue cada cópia por `gl_InstanceID`.

Função Paramétrica da Curva do Coração

O corpo é definido parametricamente para $t \in [0, 2\pi)$:

$$x(t) = 16 \operatorname{sen}^3 t, \quad (1)$$

$$y(t) = 13 \cos t - 5 \cos 2t - 2 \cos 3t - \cos 4t. \quad (2)$$

Como o espaço de recorte do OpenGL é o cubo $[-1, 1]^3$, e a curva é maior (e descentralizada), aplicamos uma escala e translação diretamente na geração dos vértices:

$$\hat{x}(t) = \frac{x(t)}{17}, \quad \hat{y}(t) = \frac{y(t) - c}{17}, \quad c = \frac{\max_t y(t) + \min_t y(t)}{2} \approx -2,5384. \quad (3)$$

Convertendo Curvas em Triângulos: GL_TRIANGLE_FAN I

Curvas fechadas amostradas em n pontos podem ser preenchidas usando a primitiva GL_TRIANGLE_FAN (Leque de Triângulos).

```
1 def leque(x, y):
2     """Empacota uma curva fechada como TRIANGLE_FAN:
3     primeiro o centro, depois a curva, e o primeiro
4     ponto repetido no fim para fechar a volta."""
5     x = np.concatenate(([0.0], x, [x[0]]))
6     y = np.concatenate(([0.0], y, [y[0]]))
7     return np.column_stack((x, y, np.zeros_like(x), np.
        ones_like(x))).astype('f4')
```

Convertendo Curvas em Triângulos:

GL_TRIANGLE_FAN II

Como o Leque funciona:

- ▶ O 1o. vértice (origem) é comum a todos os triângulos do leque.
- ▶ Os demais percorrem a borda da curva.
- ▶ O último vértice **repete o segundo** (primeiro da borda) para fechar o polígono.
- ▶ Uma curva de n amostras resulta em $n + 2$ vértices e forma n triângulos.

O leque em outras APIs

Apesar de serem úteis no OpenGL clássico, as primitivas variam: Direct3D ≥ 10 e Metal não suportam o leque; no Vulkan, é opcional. Para maior portabilidade futura, prefira TRIANGLES com EBO.

Visualização do Leque de Triângulos I

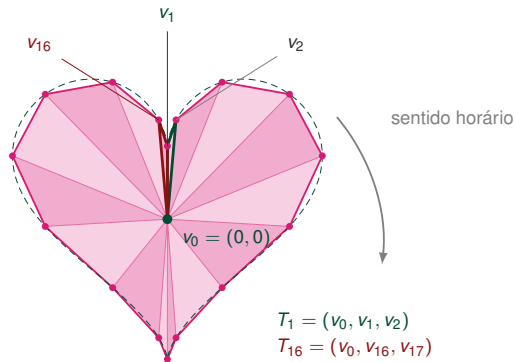


Figura: Amostragem em $n = 16$ pontos. A distância à linha tracejada (ideal) é o erro de corda. O vértice v_{17} repete v_1 para fechar a fatia.

Visualização do Leque de Triângulos II

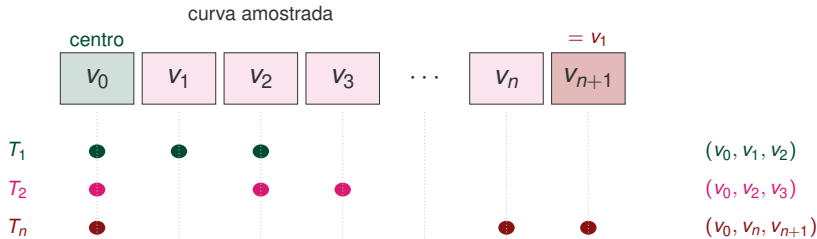


Figura: Janelas de leitura da GPU sobre os $n + 2$ vértices. v_0 participa de todos; os demais avançam um a um.

Os Shaders: Vertex Shader I

```
1 #version 400 core
2
3 layout(location = 0) in vec4 vPosition;
4
5 uniform float u_escalas;
6 uniform vec2 u_deslocamento;
7 uniform vec4 u_cor;
8
9 out vec4 v2fcolor;
10
11 void main() {
12     gl_Position = vec4(vPosition.xy * u_escalas +
13         u_deslocamento, vPosition.zw);
14
15     v2fcolor = u_cor;
```

Os Shaders: Vertex Shader II

15 | }

Listing 8: `codigos/shaders/senhor_coracao.vert`

Os Shaders: Vertex Shader III

Operações no Vertex Shader:

1. **Escala (isotrópica):** `vPosition.xy * u_escala` altera o tamanho preservando a proporção.
2. **Deslocamento:** A soma de `u_deslocamento` leva a forma à posição final.

Atenção à Ordem: A escala deve ocorrer **antes**, pois age em torno da origem do sistema local do objeto. Inverter multiplicaria o deslocamento e enviaria o objeto para longe.

Swizzling (.zw): Repassa o `z` (0.0) e `w` (1.0) recebidos intactos para a próxima etapa.

Os Shaders: Fragment Shader I

```
1 #version 400 core
2 // Usado por 06_senhor_coracao.py e por 09
   _senhor_coracao_animado.py.
3
4 in vec4 v2fcolor;
5 uniform float u_atenuacao; // 1.0 de dia, menor a noite
6 out vec4 outfragcolor;
7
8 void main() {
9     // .rgb atenua so as tres componentes de cor; .a passa
   intacto.
10    // Atenuar o alfa junto mudaria a opacidade, nao o
   brilho.
11    outfragcolor = vec4(v2fcolor.rgb * u_atenuacao, v2fcolor
   .a);
```

Os Shaders: Fragment Shader II

12 | }

Listing 9: `codigos/shaders/senhor_coracao.frag`

Os Shaders: Fragment Shader III

Diferenças para exemplos anteriores:

- ▶ O fragmento não interpola cor entre vértices distintos; ele recebe uma cor uniforme global via `u_cor`.
- ▶ O shader apenas atenua a cor baseando-se no "modo noite" e emprega *swizzling* `v2f.color.rgb` para multiplicar apenas os canais de cor, mantendo o canal alfa (`.a`) inalterado.

Código Principal: 06_senhor_coracao.py I

```
1  """
2  Senhor Coracao: uma geometria desenhada varias vezes.
3
4  Duas curvas na VRAM (o coracao e um circulo) e cinco
    desenhos por quadro. O que
5  muda de um desenho para o outro nao esta em nenhum VBO: esta
    em variaveis
6  uniformes. As teclas nao animam nada -- elas trocam o valor
    de uma uniforme, e
7  o desenho seguinte ja sai diferente.
8
9  MCCC007-23 - Computacao Grafica - UFABC
10
11  Executar: python 06_senhor_coracao.py
12  Teclas:   SETAS movem o olhar | D alterna dia/noite | ESC
    sai
13  """
```

Código Principal: 06_senhor_coracao.py II

```
14 import sys
15 from pathlib import Path
16
17 import glfw
18 import moderngl
19 import numpy as np
20
21 SHADERS = Path(__file__).parent / "shaders"
22
23 MAGENTA = (0.85, 0.10, 0.45, 1.0)
24 BRANCO_OLHO = (1.0, 1.0, 1.0, 1.0)
25 PRETO = (0.05, 0.05, 0.05, 1.0)
26 FUNDO_DIA = (0.96, 0.96, 0.94, 1.0)
27 FUNDO_NOITE = (0.05, 0.05, 0.12, 1.0)
28
29 ESCALA_CORPO = 0.62
30 ESCALA_OLHO, ESCALA_PUPILA = 0.085, 0.040
```

Código Principal: 06_senhor_coracao.py III

```
31 CENTRO_OLHO = (0.17, 0.20)    # afastamento do eixo e altura
    de cada olho
32
33 # Ate onde a pupila pode sair do centro do olho, sem escapar
    do branco.
34 LIMITE_OLHAR = ESCALA_OLHO - ESCALA_PUPILA
35 PASSO_OLHAR = LIMITE_OLHAR / 2.0
36
37
38 def leque(x, y):
39     """Empacota uma curva fechada como TRIANGLE_FAN:
    primeiro o centro, depois
40     a curva, e o primeiro ponto repetido no fim para fechar
    a volta."""
41     x = np.concatenate(([0.0], x, [x[0]]))
42     y = np.concatenate(([0.0], y, [y[0]]))
```

Código Principal: 06_senhor_coracao.py IV

```
43     return np.column_stack((x, y, np.zeros_like(x), np.  
    ones_like(x))).astype('f4')  
44  
45  
46 def curva_coracao(n=180):  
47     t = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)  
48     x = 16.0 * np.sin(t) ** 3  
49     y = (13.0 * np.cos(t) - 5.0 * np.cos(2 * t)  
50         - 2.0 * np.cos(3 * t) - np.cos(4 * t))  
51     y = y - 0.5 * (y.max() + y.min())      # centra a curva  
    na origem  
52     return leque(x / 17.0, y / 17.0)      # normaliza para  
    caber em [-1, 1]  
53  
54  
55 def curva_circulo(n=48):  
56     t = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
```

Código Principal: 06_senhor_coracao.py V

```
57     return leque(np.cos(t), np.sin(t))
58
59
60 def erro_glfw(codigo, descricao):
61     """Substitui o aviso padrão do pyGLFW: imprime código
62     e descrição de qualquer erro do GLFW em stderr"""
63     print(f"GLFW [{codigo}]: {descricao}", file=sys.stderr)
64
65
66 glfw.set_error_callback(erro_glfw)    # antes de glfw.init(),
        de proposito
67
68 if not glfw.init():
69     sys.exit("FALHA: glfw nao inicializou")
70
71 glfw.window_hint(glfw.CONTEXT_VERSION_MAJOR, 4)
72 glfw.window_hint(glfw.CONTEXT_VERSION_MINOR, 0)
```

Código Principal: 06_senhor_coracao.py VI

```
73 glfw.window_hint(glfw.OPENGL_PROFILE, glfw.  
    OPENGGL_CORE_PROFILE)  
74 glfw.window_hint(glfw.OPENGL_FORWARD_COMPAT, glfw.TRUE)  
75 # Sem a dica acima, o macOS recusa qualquer contexto 3.2+ e  
76 # create_window devolve None. E inofensiva no Windows e no  
    Linux.  
77 glfw.window_hint(glfw.SAMPLES, 4)           # quatro amostras  
    por pixel  
78 glfw.window_hint(glfw.RESIZABLE, False)    # janela quadrada:  
    ver a nota  
79  
80 janela = glfw.create_window(640, 640, "Senhor Coracao", None  
    , None)  
81 if not janela:  
82     glfw.terminate()  
83     sys.exit("FALHA: nao foi possivel criar a janela")  
84
```

Código Principal: 06_senhor_coracao.py VII

```
85 glfw.make_context_current(janela)
86 ctx = moderngl.create_context()
87
88 # SAMPLES e uma dica, nao uma garantia: o driver pode
   entregar menos amostras
89 # do que se pediu. Conferir e barato.
90 print(f"amostras por pixel obtidas: {ctx.screen.samples}")
91
92 prog = ctx.program(
93     vertex_shader=(SHADERS / "senhor_coracao.vert").
        read_text(encoding="utf-8"),
94     fragment_shader=(SHADERS / "senhor_coracao.frag").
        read_text(encoding="utf-8"),
95 )
96
97
98 def montar(vertices):
```

Código Principal: 06_senhor_coracao.py VIII

```
99     vbo = ctx.buffer(vertices.tobytes())
100     return vbo, ctx.vertex_array(prog, [(vbo, '4f', '
    vPosition')])
101
102
103 vbo_coracao, vao_coracao = montar(curva_coracao())
104 vbo_circulo, vao_circulo = montar(curva_circulo())
105
106 # Todo o estado do programa cabe em dois numeros e um
    booleano.
107 olhar_x, olhar_y = 0.0, 0.0
108 noite = False
109
110
111 def tecla(window, key, scancode, action, mods):
112     global olhar_x, olhar_y, noite
113     if action != glfw.PRESS and action != glfw.REPEAT:
```

Código Principal: 06_senhor_coracao.py IX

```
114         return
115     if key == glfw.KEY_ESCAPE:
116         glfw.set_window_should_close(window, True)
117     elif key == glfw.KEY_D:
118         noite = not noite
119     elif key in (glfw.KEY_LEFT, glfw.KEY_RIGHT, glfw.KEY_UP,
120                 glfw.KEY_DOWN):
121         dx = {glfw.KEY_LEFT: -1.0, glfw.KEY_RIGHT: 1.0}.get(
122             key, 0.0)
123         dy = {glfw.KEY_DOWN: -1.0, glfw.KEY_UP: 1.0}.get(key
124             , 0.0)
125         olhar_x += dx * PASSO_OLHAR
126         olhar_y += dy * PASSO_OLHAR
127         # O branco do olho e um CIRCULO, entao o limite tem
128         de ser no vetor.
129         # Limitar cada eixo em separado prende a pupila num
130         quadrado, e na
```

Código Principal: 06_senhor_coracao.py X

```
126         # diagonal ela escapa: o centro chega a 0.064 do
        centro do olho e a
127         # borda da pupila a 0.104, contra um raio de 0.085.
128         distancia = (olhar_x ** 2 + olhar_y ** 2) ** 0.5
129         if distancia > LIMITE_OLHAR:
130             olhar_x *= LIMITE_OLHAR / distancia
131             olhar_y *= LIMITE_OLHAR / distancia
132
133
134 glfw.set_key_callback(janela, tecla)
135
136
137 def ajustar(nome, valor):
138     """Escreve num uniforme, se ele existir.
139
140     Um uniforme que o shader deixa de usar e descartado pelo
    ligador do
```

Código Principal: 06_senhor_coracao.py XI

```
141     GLSL, e prog[nome] levantaria KeyError. Os experimentos
do capítulo
142     fazem exatamente isso -- ignorar u_cor, ignorar
u_atenuacao -- entao
143     o acesso passa todo por aqui.
144     """
145     uniforme = prog.get(nome, None)
146     if uniforme is not None:
147         uniforme.value = valor
148
149
150 def desenhar(vao, escala, deslocamento, cor):
151     """Carrega os tres uniformes daquele desenho e dispara a
chamada."""
152     ajustar('u_escala', escala)
153     ajustar('u_deslocamento', deslocamento)
154     ajustar('u_cor', cor)
```

Código Principal: 06_senhor_coracao.py XII

```
155     vao.render(moderngl.TRIANGLE_FAN)
156
157
158 while not glfw.window_should_close(janela):
159     # Uma escrita por quadro: os cinco desenhos abaixo leem
este mesmo valor.
160     ajustar('u_atenuacao', 0.35 if noite else 1.0)
161
162     ctx.clear(*(FUNDO_NOITE if noite else FUNDO_DIA))
163
164     desenhar(vao_coracao, ESCALA_CORPO, (0.0, 0.0), MAGENTA)
165
166     # A mesma geometria de circulo, quatro vezes, so mudando
os uniformes.
167     for lado in (-1.0, 1.0):
168         centro = (lado * CENTRO_OLHO[0], CENTRO_OLHO[1])
```

Código Principal: 06_senhor_coracao.py XIII

```
169         desenhar(vao_circulo, ESCALA_OLHO, centro,
BRANCO_OLHO)
170         desenhar(vao_circulo, ESCALA_PUPILA,
171                 (centro[0] + olhar_x, centro[1] + olhar_y),
PRETO)
172
173     glfw.swap_buffers(janela)
174     glfw.poll_events()
175
176     for recurso in (vao_coracao, vao_circulo, vbo_coracao,
vbo_circulo, prog):
177         recurso.release()
178     glfw.terminate()
179     print("Execucao finalizada.")
```

Listing 10: codigos/06_senhor_coracao.py

Observações de Projeto e Otimizações I

- Ordem dos Desenhos: O corpo foi desenhado antes dos olhos. Como não há teste de profundidade ativo, os últimos objetos sobrescrevem visualmente os anteriores (*Painter's Algorithm*).
- Movimento da Pupila: O limite para não sair do globo ocular é a diferença geométrica dos raios: $ESCALA_OLHO - ESCALA_PUPILA$.

Observações de Projeto e Otimizações II

O Erro `KeyError` em Variáveis Uniformes

O compilador GLSL remove qualquer variável uniforme declarada que não afete o resultado final (código morto).

- ▶ O `moderngl` mapeia apenas as variáveis **ativas** na VRAM.
- ▶ Tentar atribuir via dicionário (ex: `prog['u_atenuacao']`) lançará um `KeyError` se o shader não a utilizou.
- ▶ **Solução:** Use `prog.get('u_atenuacao', None)` para testes seguros, pois o Python tolerará a ausência da chave se você comentar linhas no GLSL durante experimentos.

Propostas de Experimentos Práticos I

1. Altere o MSAA (amostras) no `glfw.SAMPLES` (de 4 para 0, depois 16) e compare a borda.
2. Aplique *swizzling* `v2fcolor.bgr` na cor de saída do Fragment e veja o coração mudar de tom sem alterar o Python.
3. Teste `v2fcolor.rrr` na saída. Qual o resultado?.
4. Teste `vPosition.yx` no Vertex Shader. O que ocorre com os olhos e com a forma geométrica?.
5. Inverta ordem de escala e deslocamento no GLSL. Calcule no papel e comprove o erro na tela.
6. Transforme `u_escala` (que hoje é isotrópica) em um `vec2` para achatar a personagem.
7. **Desafio:** Acrescente uma boca (terceiro círculo escalonado e transladado).

A Interpolação do Rasterizador I

O rasterizador decide que fração de um fragmento pertence a cada vértice utilizando **coordenadas baricêntricas**.

Seja um ponto P dentro de um triângulo (V_1, V_2, V_3), sua posição é uma combinação linear:

$$P = \alpha V_1 + \beta V_2 + \gamma V_3, \quad \text{com} \quad \alpha + \beta + \gamma = 1 \quad \text{e} \quad \alpha, \beta, \gamma \geq 0.$$

A Geometria dos Pesos: Traçando linhas de P aos vértices, criamos três sub-triângulos. O peso de um vértice é a razão da área do pedaço **oposto** a ele pela área total:

$$\alpha = \frac{A_\alpha}{A_{\text{total}}}, \quad \beta = \frac{A_\beta}{A_{\text{total}}}, \quad \gamma = \frac{A_\gamma}{A_{\text{total}}}.$$

A Interpolação do Rasterizador II

Cálculo da Área (Determinante e Orientação): O cálculo das áreas deriva da geometria em 2D. O determinante das coordenadas avalia a área e a orientação topológica (negativo = sentido horário):

$$C = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}.$$

O Grande Trunfo: Reaproveitamento Uma vez calculados os pesos baricêntricos espaciais, o hardware utiliza os **mesmos valores** para interpolar qualquer outro atributo daquele VBO:

$$C_P = \alpha C_1 + \beta C_2 + \gamma C_3.$$

A Interpolação do Rasterizador III

Ele é agnóstico à grandeza. Interpola cegamente coordenadas, cores, texturas ou luz. Esta indiferença trará consequências físicas que abordaremos no estudo de iluminação.